アルゴリズム基礎知識 II

第5章：三目並べを作ろう

～ 線形探索法（Linear Search）の応用と勝敗判定 ～

プログラミング学習シリーズ | 講義用スライド資料

01. アルゴリズム基礎

線形探索法（リニアサーチ）とは？

線形探索法（Linear Search）とは、データ構造（配列やリスト）の先頭から順番に1つずつ目的のデータを探していく、最も基本的で直感的な探索アルゴリズムです。

アルゴリズムの特徴

- **単純明快:** 端から順に比べるだけのシンプルな仕組み。
- **前処理不要:** データが整列（ソート）されていなくても適用可能。
- **計算量:** 最悪計算量は O(N)。要素数 N に比例して試行回数が増加。

動作イメージ（配列から '7' を探す）



線形探索の基本コード（Python例）

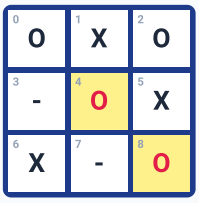
```
def linear_search(target, data_list): for index, value in enumerate(data_list): if value == target: return index # 見つかった場合、
```

02. ゲームのモデル化

三目並べの盤面と配列データ構造

三目並べ（3×3のマス目）の盤面をプログラムで扱うには、**1次元配列（要素数9）**にマッピングして管理するのが効率的です。

盤面インデックスの割り当て



データ表現ルール

- **空きマス:** `" "`（スペース）または `"-"`
- **プレイヤー1 (先手):** `"O"`
- **プレイヤー2 (後手):** `"X"`

配列表現の例:

```
board = ["O", "X", "O", " ", "O", "X", "X", " ", "O"]
```

探索の課題:「縦・横・斜めのいずれかのラインに、同じマークが3つ並んでいるか？」をどのように判定するか？ → **勝利ラインリストの線形探索**を行います。

03. アルゴリズムの応用

勝利判定における「線形探索」の適用

三目並べの勝利パターンは全部で **8通り**（横3列、縦3列、斜め2列）しかありません。
この「全8通りの勝利ライン」を順に調べていくアプローチこそが**線形探索法**です。

1. 横のチェック (3通り)

- Top: [0, 1, 2]
- Middle: [3, 4, 5]
- Bottom: [6, 7, 8]

2. 縦のチェック (3通り)

- Left: [0, 3, 6]
- Center: [1, 4, 7]
- Right: [2, 5, 8]

3. 斜めのチェック (2通り)

- Main: [0, 4, 8]
- Anti: [2, 4, 6]

判定のながれ（線形探索アルゴリズム）

1. 勝利条件のインデックスリスト（全8パターンの配列）を準備する。
2. リストの先頭（1つ目の勝利パターン）から順に探索を開始する。
3. 「パターン内の3マスすべてが同じプレイヤーのマーク（かつ空でない）」か判定する。
4. **一致した場合:** 探索を中断し、そのプレイヤーを「勝ち」として返す（探索成功）。
5. **8パターン全て不一致の場合:** 勝者なし（ゲーム継続または引き分け）。

04. 実装コード

勝利判定関数のPython実装

```
    # 勝利パターンの定義リスト（8個の直線パターン） WINNING_LINES = [ [0, 1, 2], [3, 4, 5], [6, 7, 8], # 横ライン [0, 3, 6], [1, 4, 7], [2, 5, 8],
```

> **ポイント:** `for line in WINNING_LINES:` の部分が線形探索そのものです。
> 全8行の組み合わせをループで順番に検証し、一致した時点で即座に `return`（早期脱出）します。

05. まとめと演習

本章のまとめとチャレンジ課題

本章の学習ポイント

- **線形探索法の理解:** 先頭から順番に1つずつチェックする最も基本の探索。
- **二次元→一次元変換:** 3×3の盤面を長さを9の1次元配列として表現。
- **勝利判定の簡略化:** 複雑そうなゲーム判定も「全8パターンの線形探索」に落とし込める。

次ステップへのチャレンジ

- **引き分け（Draw）の判定:** 空きマス（" "）が残っているかを線形探索で判定してみよう！
- **N×Nへの拡張:** 4×4の盤面（四目並べ）になった場合、探索パターンはどう変化するか？

おつかれさまでした！

線形探索法をマスターして、次はAIプレイヤー（思考アルゴリズム）の実装に挑戦してみましょう。